

CERTIFY low-end RISC-V node Hardware documentation

Introduction

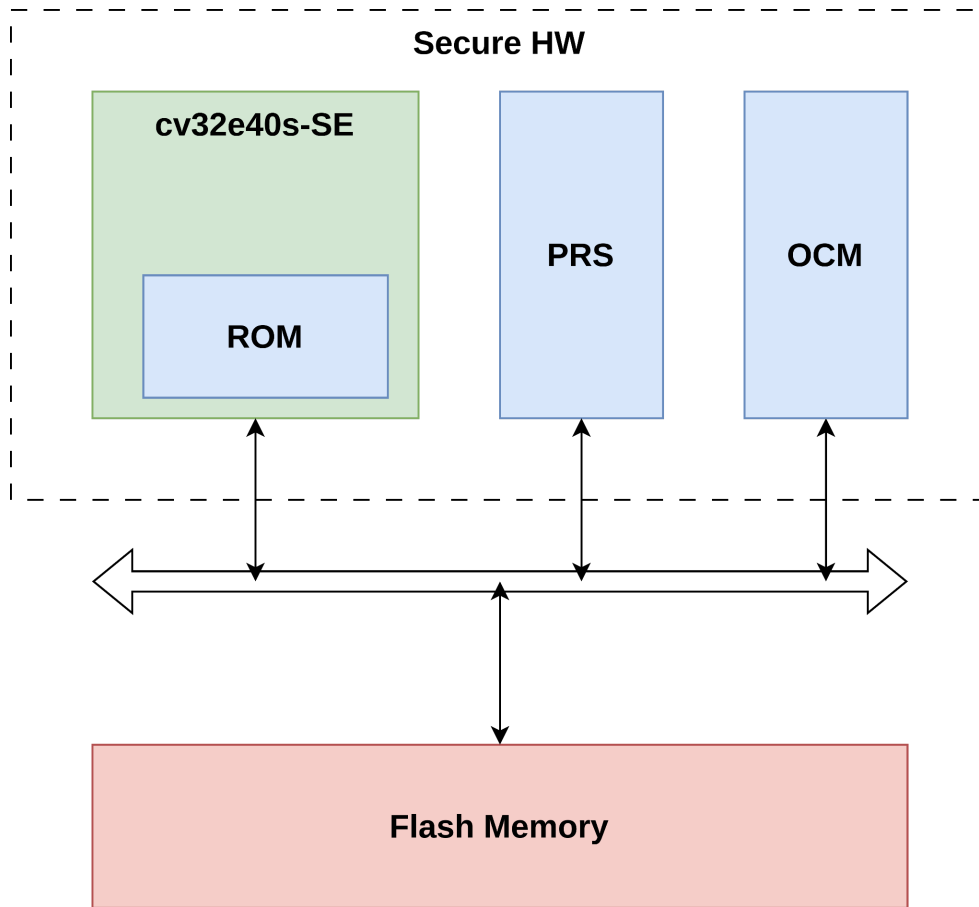
The objective of this document is to present the hardware modifications required for the CERTIFY low-end open hardware node.

The motivation for these modifications arises from the need for strong isolation between CERTIFY software services and the fact that the low-end node is an MCU-class device, providing only two execution modes (Machine and User). This limitation does not allow for the assurance of environment isolation while still enabling code execution in Machine mode. Secure booting of the device was also a key factor guiding the hardware modifications.

Summary of the modifications

The [CV32E40S](#) core is extended by adding a ROM used as a root of trust for secure boot and device root key and mud url permanent storage, and by adding a new *execution security level*. The SoC has a Secure HW perimeter that encompasses the custom core, an On Chip Memory (OCM) and a Persistent Secure Memory (PRS). All hardware inside the secure perimeter is secured against physical attacks. The OCM is used as the main memory of the device while the PRS is used to store modifiable non-volatile information.

The basic idea is that code executing with Machine mode is not anymore allowed to access any register/peripheral/memory address and additional checks will be done to assess the code security level. Only the code executing with Machine mode AND with a secure execution security level is allowed to have full control of the hardware. The extended core is named **CV32E40S-SE**. Following is an image clarifying the hardware structure:



Specification

Following is the detailed hardware specification of the platform:

1. The core CV32E40S is extended with a secure ROM used as a root of trust,
2. The core is featuring the [SMEPMP](#) extension,
3. The core is extended with a new state. This state is called **secure_lvl** and can have values from {**S**, **NS**}. **S** means secure, **NS** means non-secure. The core always boots with `secure_lvl = S`,
4. The core sends the value of `secure_lvl` to its PMPs,
5. The core `pmpXcfg` (for each X in ISA) registers are modified as follows:
 - a. Bit 6, named from now on **pmpXcfg.SL**, can be written with values 0 and 1 iff `pmpXcfg.L != 1` (unlocked configuration),
 - b. The configuration of a register `pmpXcfg` and its counterpart `pmpaddrX` is said to be **Secure Locked** if `pmpXcfg.SL == 1` and any writes to them are ignored if `secure_lvl != S`. If `pmpXcfg.A == TOR`, any writes to `pmpaddr(X-1)` are also ignored.

6. The core PMPs are modified in such a way that if `secure_lvl == S` and the execution mode is Machine, **all access is permitted**. In all other cases the PMP works normally,
7. The core ISA is extended with new instructions:
 - a. **SMENTER**. This atomically executes:
 - i. `secure_lvl <- S`
 - ii. The behavior of an ECALL:
 1. `MPIE <- MIE`
 2. `MIE <- 0`
 3. `MPP <- U/M` (according to previous privilege level)
 4. `MEPC <- PC`
 - iii. `PC <- SMENTER_HANDLER_ADDRESS` (fixed at hardware level)
 - b. **SMEXIT**. This atomically executes:
 - i. `secure_lvl <- NS`
 - ii. The behavior of an MRET:
 1. `MIE <- MPIE`
 2. Core execution mode <- MPP
 3. `MPIE <- 1`
 4. `MPP <- U`
 5. `PC <- MEPC`
8. The SoC has an OCM and a PRS. The OCM is a volatile memory, while the PRS is a non-volatile memory,
9. The SoC has a secure perimeter that includes the core, the OCM, and the PRS. The hardware in the secure perimeter maintains the confidentiality and integrity of the data/software it holds.

Execution modes and security

The following diagram highlights the interaction between software components executing at different machine modes (User and Machine) and security (Secure and Non-Secure) levels:

